

CSE160 Homework 2

Elijah Hantman

April 18, 2026

Part 1

1. Using the labels Translate, Rotation, Scale, or Reflect, label the transformation of point p when multiplied by matrix M ($p' = Mp$):

(a)

$$M = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

This is a rotation by 90 degrees.

(b)

$$M = \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

This is another rotation, by 45 degrees.

(c)

$$M = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This is a non uniform scaling.

(d)

$$M = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

This is a reflection about e_2 , meaning a vector $v = ae_1 + be_2$ will be mapped to $v = -ae_1 + be_2$.

(e)

$$M = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$$

In a 2D projective space, this would correspond to a translation by $(3, 2)$.

2. Find the matrix for a 120° rotation about the axis defined by the vector $r = (1, 1, 0)$.

The matrix can be derived via the product of a rotation to put the axis onto one of the bases, then a basis aligned rotation, followed by the inverse of the original rotation.

Using a tool to calculate the rotation matrix we get:

$$M = \begin{bmatrix} 0.2500000 & 0.7500000 & 0.6123725 \\ 0.7500000 & 0.2500000 & -0.6123725 \\ -0.6123725 & 0.6123725 & -0.5000000 \end{bmatrix}$$

Alternative methods involve using quaternions, then converting to a matrix, or you could decompose the rotation using euler angles. Ie: first doing a rotation to put the axis on the basis, then doing a rotation, then undoing it.

3. Match the following 2D homogeneous matrices to the transformations in the image:

1 goes with matrix b.
2 goes with matrix a
3 goes with matrix c

4. Describe a sequence of Translate(x,y), Rotate(degrees), and/or Scale (by a value of) that when multiplied describe the below transformation to go from the 'Before' to 'After'.

Translate (-1, 0)
Rotate(45)

I can't tell if the after image has a scale applied, so I didn't include that, but if I did, it would have to happen first.

5. (a) Describe (using a sequence of (3 x 3) matrix multiplications) the transformations needed to transform the triangle from A to B in the figure below:

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} 1 & 0 & -7 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Something to note is that I do this in 2 steps. First I scale and reflect the triangle, then I translate it into position. I also do not forget that scaling far from the origin causes the position to shift.

- (b) Using the transformations found in part (a), multiply the following points with the matrix.

$$p_1 = \begin{bmatrix} 4 \\ 1 \end{bmatrix} \quad p_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$p_1 = (4, 1)$$

$$\rightarrow (8, -1)$$

$$\rightarrow (1, 4)$$

$$p_2 = (4, 3)$$

$$\rightarrow (8, -3)$$

$$\rightarrow (1, 2)$$

Part 2

1. Define uniform, attribute and varying variables

Uniform variables are variables which are constant between vertex shader invocations.

On the other hand attribute and varying variables contain the data from vertices, and interpolated vertex shader outputs respectively.

In more modern GLSL uniforms still exist since the idea of simply making a single value accessible across all shader executions is very useful.

On the other hand in order to make the API more descriptive attribute and varying variables are simply labeled as "in" and "out" variables.

This means that the variables are more clearly defined in regards to the shader they appear in. For example, a vertex shader will output to an "out" variable, which would be labeled varying in GLSL 1.

And a fragment shader will take as input an "in" variable rather than a varying variable.

2. What type is the shader and describe its function?

```
(a) attribute vec2 a_Position;
    main() {
        gl_Position = vec4(0.5 * a_Position, 0.0, 1.0);
    }
```

This is a vertex shader. We can tell by the use of "attribute" to declare vertex data, as well as "gl_Position" which is a vertex shader builtin which is used to set the position of the vertex for rasterization.

```
(b) main() {
    gl_FragColor = vec4(0.0, 0.0, 1.0, 1.0);
}
```

This is a fragment shader. This is obvious by the usage of "gl_FragColor" as well as the lack of attribute declarations.

3. Using linear interpolation, calculate the color of the following points that are on the line defined by the 2D points $p_1 = [3, 5]$ with color $[255, 0, 0]$ and $p_2 = [10, 5]$ with color $[0, 0, 255]$.

(a) $[4.8, 5]$

$$[4.8, 5] = t[10, 5] + (1 - t)[3, 5]$$

$$[4.8, 5] = tp_2 + p_1 - tp_1$$

$$[4.8, 5] = t(p_2 - p_1) + p_1$$

$$[1.8, 0] = t(p_2 - p_1)$$

$$[1.8, 0] = t([7, 0])$$

$$t \approx 0.25$$

Now we lerp the colors.

$$c = t(c_2 - c_1) + c_1$$

$$c = t([-255, 0, 255]) + [255, 0, 0]$$

$$c \approx [192, 0, 64]$$

(b) $[7.5, 5]$

Since most of the steps in the previous problem didn't require using the point we can skip most of the steps.

$$[4.5, 0] = t([7, 0])$$

$$t \approx 0.64$$

Now to lerp the colors

$$c = t([-255, 0, 255]) + [255, 0, 0]$$

$$c \approx [92, 0, 163]$$

(c) $[9, 5]$

$$[6, 0] = t([7, 0])$$

$$t \approx 0.86$$

Next the colors:

$$c = t([-255, 0, 255]) + [255, 0, 0]$$

$$c \approx [36, 0, 219]$$

4. Using barycentric coordinates, calculate the color for each of the following points inside the triangle defined by points $p_1 = [0, 0, 0]$, $p_2 = [3, 5, 0]$ and $p_3 = [6, 0, 0]$ with respective RGB colors $c_1 = [255, 0, 0]$, $c_2 = [0, 255, 0]$, $c_3 = [0, 0, 255]$.

(a) $[1, 1, 0]$

To calculate barycentric coordinates you do,

$$v_1 = p_2 - p_1$$

$$v_2 = p_3 - p_1$$

$$v_3 = [1, 1, 0] - p_1$$

$$A = |v_1 \times v_2|$$

$$b_2 = |v_3 \times v_2|$$

$$b_3 = |v_3 \times v_1|$$

$$b_1 = 1 - (b_2 + b_3)$$

Then to get the color we do $b_1 c_1 + b_2 c_2 + b_3 c_3$.

$$b = [11/15, 1/5, 1/15]$$

$$c = [187, 51, 17]$$

(b) $[3, 4, 0]$

$$b = [1/10, 4/5, 1/10]$$

$$c = [26, 204, 26]$$

(c) $[5, 0.25, 0]$.

$$b = [0.14, 0.05, 0.81]$$

$$c = [36, 13, 207]$$

5. Calculate the RGB color value for point P in the diagram below using bilinear interpolation. Show the intermediate values that you compute for interpolating along one of the two axes. The colors of the points A, B, C and D are the following:

$$A = \text{RGB}(0, 0, 0) \quad B = \text{RGB}(200, 200, 200)$$

$$C = \text{RGB}(200, 0, 0) \quad D = \text{RGB}(200, 0, 0)$$

$$A = (14, 21) \quad B = (15, 21)$$

$$C = (14, 20) \quad D = (15, 20)$$

$$P = (14, 75, 20.5)$$

So we can instantly see that the lerp $C \rightarrow D$ will always give the result $\text{RGB}(200, 0, 0)$. This follows since $C = D$.

Next we need to interpolate $A \rightarrow B$.

$$14.75 = t(15 - 14) + 14$$

$$0.75 = t$$

Then the color.

$$c = 0.75(\text{RGB}(200, 200, 200) - \text{RGB}(0, 0, 0)) + \text{RGB}(0, 0, 0)$$

$$c = \text{RGB}(150, 150, 150)$$

Next we interpolate vertically. Since the quad is axis aligned, we can just use the y coordinate.

$$20.5 = t(21 - 20) + 20$$

$$0.5 = t$$

Next the interpolation

$$c = 0.5(\text{RGB}(150, 150, 150) - \text{RGB}(200, 0, 0)) + \text{RGB}(200, 0, 0)$$

$$c = 0.5(RGB(-50, 150, 150)) + RGB(200, 0, 0)$$

$$c = RGB(175, 75, 75)$$

6. Given a polygon with vertices P1, P2, P3, each represented by a 3D vector $\begin{bmatrix} x & y & z \end{bmatrix}$, what formula calculates the face's normal?

The normal is the vector facing directly perpendicular to the face. Assuming the winding order is $P1, P2, P3$.

$$v_1 = P2 - P1$$

$$v_2 = P3 - P1$$

$$p = v_1 \times v_2$$

$$n = \frac{p}{|p|}$$

Since the cross product is defined as the vector perpendicular to v_1 and v_2 . However the magnitude is the product $|v_1||v_2|\sin(\theta)$. So to get a normal with unit length we need to normalize the vector.

Part 3

1. Given a point $p = (x, y, z)$ and the point $p' = (x', y', z')$ where $p' = (x', y', z')$ is the point $p = (x, y, z)$ translated by T_x, T_y, T_z :

- (a) Show the equations to compute p' from p

$$x' = x + T_x$$

$$y' = y + T_y$$

$$z' = z + T_z$$

- (b) Show the 4×4 transformation matrix which takes p to p' .

$$M = \begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. Given a point $p(x, y, z)$ and the point $p'(x', y', z')$, where $p'(x', y', z')$ is the β degree rotated point of $p(x, y, z)$ around the z-axis:

- (a) Show the equation to compute p' from p .

$$\begin{aligned}x' &= x\cos(\beta) - y\sin(\beta) \\y' &= x\sin(\beta) + y\cos(\beta) \\z' &= z\end{aligned}$$

- (b) Show the 4×4 transformation matrix.

$$M = \begin{bmatrix} \cos(\beta) & -\sin(\beta) & 0 & 0 \\ \sin(\beta) & \cos(\beta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3. Given a point $p(x, y, z)$ and the point $p'(x', y', z')$, where $p'(x', y', z')$ is the point $p(x, y, z)$ scaled by S_x, S_y, S_z :

- (a) Show the equation to compute p' from p .

$$\begin{aligned}x' &= xS_x \\y' &= yS_y \\z' &= zS_z\end{aligned}$$

- (b) Show the 4×4 transformation matrix.

$$M = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4. What is a model matrix? Write a simple vertex shader that uses a model matrix. Explain your shader.

A model matrix is a matrix which transforms objects from a local space where they contain their own origin, to world space where all objects are positioned relative to each other.

In practice this means a model matrix is unique per object, and can do transformations before the camera and view matrices transform the scene into clip space.

```
#version 300 es
in vec3 pos;
```

```
uniform mat4 model;

main() {
    gl_Position = model * vec4(pos, 1.0);
}
```

This is a super simple vertex shader. It uses GLSL 2 which is why it has the version declaration at the top.

Basically it reads in a vec3 from the vertex data, and a uniform model matrix. Then it sets the position to the model matrix applied to the position via left multiplication.

This shader is one what would be used via a draw call per object with the model matrix being set per object. I also use a common pattern of converting the position to a vec4, rather than storing it as a vec3 since we always want the w component to be 1.